

Spring AI: Module 05

RAG Part 1: Embeddings, Vector Stores & Document ETL

Enterprise Java AI Engineering

1. What is RAG? (Retrieval-Augmented Generation)

- LLMs have knowledge cutoffs and lack private enterprise domain context.
- **RAG** bridges this gap:
 1. Retrieve relevant company knowledge chunks from a vector database.
 2. Inject those chunks into the prompt context.
 3. Synthesize a fully grounded, hallucination-free answer.

2. Dense Vector Embeddings

- Text strings are converted into dense numerical vectors of fixed dimension (e.g. 384, 768, 1536 floats).
- Semantically similar phrases cluster closely in high-dimensional vector space:
 - `distance(king, queen) ~ distance(man, woman)`
 - Measured via **Cosine Similarity** or Euclidean Distance.

3. Spring AI Document ETL Architecture

```
[Raw Document] -> [DocumentReader]
                  -> [TokenTextSplitter]
                  -> [EmbeddingModel]
                  -> [VectorStore (pgvector / Simple)]
```

- **Document** : Carries string text and key-value metadata.
- **TokenTextSplitter** : Splits long documents respecting token boundaries to avoid truncating words.

4. Ingestion Code Example

```
Document rawDoc = new Document(policyText, Map.of(
    "department", "Security",
    "version", "2026.1"
));

List<Document> chunks = tokenTextSplitter.apply(List.of(rawDoc));
vectorStore.accept(chunks);
```

5. Similarity Search with Thresholds

```
SearchRequest request = SearchRequest.builder()  
    .query("How to connect to VPN?")  
    .topK(3)  
    .similarityThreshold(0.70)  
    .filterExpression("department == 'Security'")  
    .build();  
  
List<Document> relevantChunks = vectorStore.similaritySearch(request);
```

6. Synthesizing Grounded Answers

```
String renderedPrompt = promptTemplate.render(Map.of(  
    "context", retrievedSnippets,  
    "question", userQuestion  
));
```

```
String response = chatClient.prompt()  
    .user(renderedPrompt)  
    .call()  
    .content();
```

7. Key Takeaways

1. Never dump massive PDFs whole into context; chunk them accurately.
2. Enrich documents with metadata for pre-filtering.
3. Enforce similarity cutoffs to prevent irrelevant hallucinations.